

LINKED LIST

MASTER NOTES

FAANG Placement • LeetCode • Codeforces • OA Rounds

10 Core Patterns	Complete Dry Runs	C++ Code Only
Brute → Optimal	Interview Mode	CP Contest Mode
Common Traps	Edge Cases	Similar Problems

PATTERNS COVERED

P01 Fast & Slow Pointer	P06 Merge Technique
P02 Reverse Linked List	P07 K-Group Reversal
P03 Dummy Node	P08 Linked List + Stack
P04 Two Pointer / Nth from End	P09 Recursion on LL
P05 Floyd Cycle Detection	P10 In-Place Manipulation

Structured for Pattern Recognition • Interview Communication • Contest Thinking

■ QUICK CHEAT SHEET — When To Use Which Pattern

Pattern	Trigger Keywords	Key Idea	TC / SC
Fast & Slow Pointer	Cycle, Middle, Palindrome	Two speeds on same list	$O(N)$ / $O(1)$
Reverse LL	Reverse, Palindrome check, K-group	Rewire .next pointers	$O(N)$ / $O(1)$
Dummy Node	Remove node, Merge, Insert at head	Avoid head edge case	$O(N)$ / $O(1)$
Two Pointer / Nth End	Kth from end, Remove Nth	Gap of K between pointers	$O(N)$ / $O(1)$
Floyd Cycle	Detect loop, Find start, Loop length	Slow+fast math magic	$O(N)$ / $O(1)$
Merge Technique	Merge sorted, Sort LL	Like merge sort merge step	$O(N)$ / $O(1)$
K-Group Reversal	Reverse in K groups	Count K + reverse chunk	$O(N)$ / $O(1)$
LL + Stack	Palindrome, Reverse print, Nested	LIFO order on nodes	$O(N)$ / $O(N)$
Recursion on LL	Reverse, Print reverse, Sum	Trust base case + return	$O(N)$ / $O(N)$ stack
In-Place Manip	Reorder, Partition, Interleave	Find mid + detach + merge	$O(N)$ / $O(1)$

■ Golden Decision Rules

30-Second Pattern Recognition Rules

- See **cycle / loop / infinite** → Floyd's algorithm (P05)
- See **middle of list** → Fast & Slow pointer (P01)
- See **reverse any portion** → Reverse LL pattern (P02), K-group (P07)
- See **Kth from end / remove Nth** → Two pointer with gap K (P04)
- See **merge two sorted lists** → Merge technique with dummy node (P06 + P03)
- See **palindrome check** → Find mid + reverse second half (P01 + P02)
- See **reorder / interleave** → Find mid + reverse + merge (P10)
- See **recursion hint** → Trust the recursion, define base case clearly (P09)
- Interviewer says 'O(1) space' → Eliminate stack/array approaches immediately
- Constraints $N \leq 10^5 \rightarrow O(N \log N)$ or $O(N)$ needed. $N \leq 10^3 \rightarrow O(N^2)$ OK

■ FAANG Company Frequency

Company	Most Asked Patterns	Frequency
Google	Floyd Cycle, K-Group, Recursion LL	■■■■■
Meta	Reverse LL, Merge, Reorder List	■■■■■
Amazon	Two Pointer, Dummy Node, Merge	■■■■■
Microsoft	Reverse LL, Fast&Slow, Floyd	■■■■
Uber	Cycle Detection, LRU Cache (LL+Map)	■■■■
Adobe	In-Place Manip, Palindrome LL	■■■
Atlassian	Merge, Reverse LL, Two Pointer	■■■

Pattern 01: Fast & Slow Pointer (Tortoise & Hare)

Metric	Rating
FAANG Frequency	■■■■■ Very High
LeetCode Importance	■■■■■ Essential
Codeforces Usefulness	■■■ Moderate
Companies	Google, Meta, Amazon, Microsoft, Apple, Uber
Difficulty	Easy → Medium
Problems	LC 141, 142, 876, 234, 143, 457

■ Why This Pattern Exists

Imagine two people running on a circular track — one fast, one slow. If the track has a loop, the fast one will eventually lap the slow one and they will meet. If there is no loop, the fast one reaches the end first. This is exactly what this pattern exploits.

When to think of this: Anytime you see the word **cycle**, **loop**, **middle**, **palindrome** in a linked list problem.

■ Brain Training Questions

Think Before You Code

- What happens if the list has only 1 node? Does `slow==fast` immediately?
- Why does fast move 2 steps and slow move 1? What if fast moves 3 steps?
- If there's a cycle, will slow and fast ALWAYS meet? (Yes — prove it!)
- For finding middle: if N is even, which middle do we get — left or right?
- Can we solve 'find middle' without counting nodes first?

■ Dry Run — Find Middle of Linked List

```
List: 1 -> 2 -> 3 -> 4 -> 5 -> NULL
```

```
Init: slow = 1, fast = 1
```

```
Step 1: slow = slow.next = 2
```

```
fast = fast.next.next = 3
```

```
State: slow=2, fast=3
```

```
Step 2: slow = slow.next = 3
```

```
fast = fast.next.next = 5
```

```
State: slow=3, fast=5
```

```
Step 3: fast.next == NULL -> STOP
```

```
Middle = slow = 3 ✓  
  
--- Even List: 1 -> 2 -> 3 -> 4 -> NULL ---  
Init: slow=1, fast=1  
Step1: slow=2, fast=3  
Step2: slow=3, fast=NULL -> STOP  
Middle = 3 (second middle) ✓
```

■ Dry Run — Cycle Detection

```
List: 1 -> 2 -> 3 -> 4 -> 5 -> (back to 3)  
  
Init: slow=1, fast=1  
Step1: slow=2, fast=3  
Step2: slow=3, fast=5  
Step3: slow=4, fast=4 <-- MEET! Cycle detected ✓  
  
Key Insight: In a cycle of length L, fast gains 1 step/iteration.  
So they meet in O(L) steps = O(N) total.
```

■ C++ Code — Find Middle

```
// LC 876 - Middle of Linked List  
ListNode* middleNode(ListNode* head) {  
    ListNode* slow = head;  
    ListNode* fast = head;  
  
    while (fast != nullptr && fast->next != nullptr) {  
        slow = slow->next;        // move 1 step  
        fast = fast->next->next;   // move 2 steps  
    }  
  
    return slow; // slow is at middle  
}  
  
// Time: O(N) Space: O(1)
```

■ C++ Code — Cycle Detection (LC 141)

```
bool hasCycle(ListNode* head) {
    ListNode* slow = head;
    ListNode* fast = head;
    while (fast != nullptr && fast->next != nullptr) {
        slow = slow->next;
        fast = fast->next->next;
        if (slow == fast) return true; // cycle!
    }
    return false;
}
// Time: O(N) Space: O(1)
```

■ Common Mistakes / Traps

- `fast = fast->next->next` crashes if `fast->next` is NULL — always check BOTH in while condition
- For even-length list: slow lands on SECOND middle — know this for palindrome check
- Don't initialize `fast = head->next` — keep both at head for clean logic
- Cycle detection: check `slow==fast` AFTER moving both, not before

■ Interview Tips

- Interviewer follow-up 1: 'How do you find the START of the cycle?' → Floyd's (P05)
- Interviewer follow-up 2: 'Check if linked list is palindrome' → Find mid + reverse second half
- Always say: 'I'll use fast and slow pointers to avoid O(N) space' — shows optimization mindset
- What makes it rejected: checking `fast==slow` before moving pointers (both start at head, instant false positive)

■ CP / Contest Notes

- In CP: Often disguised as 'functional graph cycle detection' — same algorithm, different structure
- 30-sec ID: Any 'loop in sequence' or 'repeated pattern' → think fast-slow
- Constraint trick: N up to 10^5 → O(N) needed → fast-slow perfect fit

■ Similar Problems

#	Problem	Difficulty	Pattern
1	141. Linked List Cycle	Easy	Fast & Slow
2	876. Middle of the Linked List	Easy	Fast & Slow
3	234. Palindrome Linked List	Easy	Fast&Slow; + Reverse
4	457. Circular Array Loop	Medium	Fast & Slow
5	143. Reorder List	Medium	Fast&Slow; + Reverse + Merge

6	2095. Delete Middle Node	Medium	Fast & Slow
---	--------------------------	--------	-------------

Pattern 02: Reverse Linked List

Metric	Rating
FAANG Frequency	★★★★★ Highest
LeetCode Importance	★★★★★ Must Know
Codeforces Usefulness	★★★★ High
Companies	Every FAANG company — Most common LL question
Problems	LC 206, 92, 25, 234, 2074, 2130

■ Intuition

You have a chain: $A \rightarrow B \rightarrow C \rightarrow D$. You want $D \rightarrow C \rightarrow B \rightarrow A$. The trick is: while walking forward, redirect each arrow backward. You need 3 pointers: **prev** (what came before), **curr** (what you're fixing), **next** (save next before breaking the link).

■ Brain Training Questions

Think Before You Code

- What do you do with next pointer BEFORE reversing curr?
- What is prev initialized to? (NULL — the new tail's next should be NULL)
- After the loop, which pointer holds the new head?
- What if head is NULL or single node? Does your code handle it?
- For reverse between positions [m, n]: what do you connect at the ends?

■ Dry Run — Iterative Reverse

```
List: 1 -> 2 -> 3 -> NULL
Init: prev=NULL, curr=1
Iteration 1:
next = curr->next = 2 (save next)
curr->next = prev = NULL (reverse arrow)
prev = curr = 1 (advance prev)
curr = next = 2 (advance curr)
State: NULL <- 1 2 -> 3 -> NULL

Iteration 2:
next = 3
curr->next = prev = 1 (2 now points to 1)
prev = 2, curr = 3
```



```

State: NULL <- 1 <- 2 3 -> NULL

Iteration 3:
next = NULL
curr->next = prev = 2
prev = 3, curr = NULL
State: NULL <- 1 <- 2 <- 3

Loop ends (curr==NULL). New head = prev = 3 ✓

Result: 3 -> 2 -> 1 -> NULL

```

■ C++ Code — Iterative Reverse (LC 206)

```

ListNode* reverseList(ListNode* head) {
    ListNode* prev = nullptr;
    ListNode* curr = head;
    while (curr != nullptr) {
        ListNode* next = curr->next; // save next before breaking link
        curr->next = prev;           // reverse the arrow
        prev = curr;                // advance prev
        curr = next;                // advance curr
    }
    return prev; // prev is the new head
}

// Time: O(N) Space: O(1)

```

■ C++ Code — Recursive Reverse

```

ListNode* reverseList(ListNode* head) {
    // Base case: empty or single node
    if (head == nullptr || head->next == nullptr)
        return head;

    // Reverse the rest of the list
    ListNode* newHead = reverseList(head->next);

    // Make next node point back to current
    head->next->next = head;
    head->next = nullptr; // current becomes new tail
    return newHead;
}

// Time: O(N) Space: O(N) – recursion stack
// Note: Iterative preferred in interviews for O(1) space

```

■ C++ Code — Reverse Between [left, right] (LC 92)

```

ListNode* reverseBetween(ListNode* head, int left, int right) {
    ListNode dummy(0, head);
    ListNode* pre = &dummy;

    // Move pre to node just before 'left'
    for (int i = 0; i < left - 1; i++)
        pre = pre->next;

    ListNode* curr = pre->next;
    // Reverse (right - left) times using 'insert at front' trick
    for (int i = 0; i < right - left; i++) {
        ListNode* next = curr->next;
        curr->next = next->next;    // remove next from chain
        next->next = pre->next;    // insert next at front of sublist
        pre->next = next;
    }
    return dummy.next;
}

// Time: O(N) Space: O(1)

```

■ Common Mistakes / Traps

- ALWAYS save curr->next BEFORE doing curr->next = prev — classic beginner error
- After loop: new head is prev, NOT curr (curr is NULL)
- Recursive: O(N) stack space — mention this to interviewer
- LC 92: Don't try to find and re-reverse; use the 'insert at front' one-pass trick

■ Interview Tips

- Always draw the pointer rewiring on paper before coding — interviewers love this
- Follow-up: 'Reverse in groups of K' → Pattern 07 (K-Group Reversal)
- Follow-up: 'Can you do it in one pass?' → Yes, the insert-at-front trick for LC 92
- Rejected if: you use a stack or array (not O(1) space)

■ Similar Problems

#	Problem	Difficulty	Pattern
1	206. Reverse Linked List	Easy	Reverse LL
2	92. Reverse Linked List II	Medium	Reverse LL (partial)
3	25. Reverse Nodes in K-Group	Hard	K-Group Reversal
4	234. Palindrome Linked List	Easy	Fast&Slow; + Reverse
5	2074. Reverse Nodes in Even Groups	Medium	Reverse LL
6	24. Swap Nodes in Pairs	Medium	Reverse LL

Pattern 03: Dummy Node (Sentinel)

Metric	Rating
FAANG Frequency	■■■■■ Essential Technique
LeetCode Importance	■■■■■ Used in 70% of LL problems
Codeforces Usefulness	■■■■ High
Companies	Amazon, Google, Microsoft — appears as helper in almost all LL problems
Problems	LC 2, 21, 82, 83, 19, 86, 23, 203

■ Intuition

The problem with linked lists: the HEAD can change. If you delete the head, or insert before the head, your return pointer is wrong. Solution: create a fake 'dummy' node that points to the real head. Now head changes don't affect you — you always return dummy->next.

Rule: Any time a problem might delete or change the head node — use a dummy. Any time you're building a new list from scratch — use a dummy as your builder anchor.

■ Dry Run — Remove All Occurrences (LC 203)

```
List: 1 -> 2 -> 6 -> 3 -> 4 -> 5 -> 6 -> NULL val=6
dummy(0) -> 1 -> 2 -> 6 -> 3 -> 4 -> 5 -> 6 -> NULL
curr = dummy

Step: curr->next = 1 (not 6) → advance curr to 1
Step: curr->next = 2 (not 6) → advance curr to 2
Step: curr->next = 6 == val → curr->next = curr->next->next = 3
(skip the 6, don't advance curr)
Step: curr->next = 3 (not 6) → advance to 3
Step: curr->next = 4 → advance to 4
Step: curr->next = 5 → advance to 5
Step: curr->next = 6 == val → curr->next = NULL
Step: curr->next = NULL → stop

Result: dummy->next = 1 -> 2 -> 3 -> 4 -> 5 -> NULL ✓
```

■ C++ Code — Remove Elements (LC 203)

```

ListNode* removeElements(ListNode* head, int val) {
    ListNode dummy(0); // dummy node, next defaults to nullptr
    dummy.next = head;
    ListNode* curr = &dummy;
    while (curr->next != nullptr) {
        if (curr->next->val == val) {
            curr->next = curr->next->next; // skip the node
        } else {
            curr = curr->next; // only advance if NOT removed
        }
    }
    return dummy.next; // real head (may have changed)
}
// Time: O(N) Space: O(1)

```

■ C++ Code — Merge Two Sorted Lists (LC 21)

```

ListNode* mergeTwoLists(ListNode* l1, ListNode* l2) {
    ListNode dummy(0);
    ListNode* tail = &dummy; // tail of result list
    while (l1 && l2) {
        if (l1->val <= l2->val) {
            tail->next = l1;
            l1 = l1->next;
        } else {
            tail->next = l2;
            l2 = l2->next;
        }
        tail = tail->next;
    }
    // Attach remaining nodes
    tail->next = l1 ? l1 : l2;
    return dummy.next;
}
// Time: O(N+M) Space: O(1)

```

■ Common Mistakes / Traps

- Memory: `ListNode dummy(0)` on stack is fine for interviews. Don't heap-allocate unnecessarily.
- Always return `dummy.next` — NOT `dummy` (`dummy` is the fake node!)
- Don't advance `curr` after deletion — the new `curr->next` might also need deletion
- LC 82 (remove duplicates II): tricky because you skip ALL copies, not just one

■ Interview Tips

- Dummy node = 'make head changes safe' → say this phrase in interview

-
- It's not a pattern alone — it's a technique USED INSIDE other patterns
 - Follow-up: 'What if you cannot modify nodes?' → Same approach, just redirect pointers

Pattern 04: Two Pointer — Kth / Nth From End

Metric	Rating
FAANG Frequency	■■■■■ Very High
LeetCode Importance	■■■■■ Essential
Codeforces Usefulness	■■■ Moderate
Companies	Amazon, Google, Microsoft, Meta
Problems	LC 19, 61, 2095, 1721

■ Intuition

You don't know the length of the list. But you want the Nth node from the END. Trick: move one pointer N steps ahead. Now move both pointers together. When the front pointer hits NULL, the back pointer is exactly at the Nth node from end. It's like keeping a 'gap' of N between two runners.

■ Dry Run — Remove Nth From End (LC 19)

```
List: 1 -> 2 -> 3 -> 4 -> 5 -> NULL, n=2

dummy -> 1 -> 2 -> 3 -> 4 -> 5 -> NULL
fast = dummy, slow = dummy

Move fast n+1=3 steps (so slow stops at node BEFORE target):
fast -> 1 -> 2 -> 3 (fast is now at node 3)

Move both until fast==NULL:
fast=4, slow=1
fast=5, slow=2
fast=NULL, slow=3

slow is now at node 3 (one before the 2nd from end = node 4)
slow->next = slow->next->next => skip node 4

Result: 1 -> 2 -> 3 -> 5 -> NULL ✓
```

■ C++ Code — Remove Nth From End (LC 19)

```

ListNode* removeNthFromEnd(ListNode* head, int n) {
    ListNode dummy(0, head);
    ListNode* fast = &dummy;
    ListNode* slow = &dummy;

    // Move fast n+1 steps ahead (slow stops ONE before target)
    for (int i = 0; i <= n; i++)
        fast = fast->next;

    // Move both until fast hits end
    while (fast != nullptr) {
        fast = fast->next;
        slow = slow->next;
    }

    // slow->next is the node to remove
    slow->next = slow->next->next;
    return dummy.next;
}

// Time: O(N) Space: O(1) One-pass!

```

■ Common Mistakes / Traps

- Move fast $n+1$ steps (not n) so slow stops BEFORE the target node
- Use dummy node so even removing the head node works ($n == \text{list length}$)
- Check: what if $n > \text{list length}$? (problem guarantees it won't, but ask interviewer)
- LC 61 Rotate List: be careful about cycles — close the list into a ring, then break

■ Interview Tips

- This is a ONE-PASS $O(N)$ solution — highlight this over the naive two-pass approach
- Naive: find length L , then walk $L-n$ steps. That's two passes. This is one pass.
- Follow-up: 'Find the Kth element from end WITHOUT removing' → same, just return slow->next

■ Similar Problems

#	Problem	Difficulty	Pattern
1	19. Remove Nth Node From End	Medium	Two Pointer Gap
2	61. Rotate List	Medium	Two Pointer + Cycle
3	2095. Delete Middle of LL	Medium	Fast & Slow
4	1721. Swapping Nodes in LL	Medium	Two Pointer

Pattern 05: Floyd's Cycle Detection — Find Cycle Start

Metric	Rating
FAANG Frequency	■■■■■ Google Favourite
LeetCode Importance	■■■■■ Must Know
Codeforces Usefulness	■■■■ High
Companies	Google, Amazon, Microsoft, Adobe
Problems	LC 141, 142, 287, 457

■ The Math Behind Floyd's Algorithm

Let: F = distance from HEAD to cycle start, C = cycle length, K = distance from cycle start to meeting point.

When slow and fast meet: slow traveled $F+K$, fast traveled $F+K+nC$ (n full cycles).

Since fast is 2x speed: $2(F+K) = F+K+nC \rightarrow F+K = nC \rightarrow F = nC-K$.

Key insight: $F = nC - K$ means: if you start one pointer from HEAD and one from MEETING POINT, and both move 1 step at a time, they meet exactly at the CYCLE START!

■ Dry Run — Find Cycle Start (LC 142)

```
List: 1 -> 2 -> 3 -> 4 -> 5 -> 6 -> (back to 3)
0 1 2 3 4 5
Cycle start = node 3 (index 2)

PHASE 1: Find meeting point
slow: 1->2->3->4->5->6->3->4->5->6->3->4->...
fast: 1->3->5->3->5->3->5->...
They meet at node 5 (for example).

PHASE 2: Find cycle start
ptr1 = head (node 1)
ptr2 = meeting point (node 5)

Move both 1 step:
ptr1=2, ptr2=6
ptr1=3, ptr2=3 <-- MEET! This is the cycle start ✓
```

■ C++ Code — Find Cycle Start (LC 142)


```

ListNode* detectCycle(ListNode* head) {
    ListNode* slow = head;
    ListNode* fast = head;

    // Phase 1: Detect if cycle exists and find meeting point
    while (fast != nullptr && fast->next != nullptr) {
        slow = slow->next;
        fast = fast->next->next;
        if (slow == fast) {
            // Phase 2: Find cycle entry point
            ListNode* ptr = head;
            while (ptr != slow) {
                ptr = ptr->next;
                slow = slow->next;
            }
            return ptr; // cycle start
        }
    }
    return nullptr; // no cycle
}

// Time: O(N) Space: O(1) — No HashSet needed!

```

■ Brute Force — Using HashSet

```

// Brute: store visited nodes in a set
ListNode* detectCycle(ListNode* head) {
    unordered_set<ListNode*> visited;
    ListNode* curr = head;
    while (curr != nullptr) {
        if (visited.count(curr)) return curr;
        visited.insert(curr);
        curr = curr->next;
    }
    return nullptr;
}

// Time: O(N) Space: O(N) -- Not optimal!
// Interviewer will ask: 'Can you do O(1) space?' -> Floyd's

```

■ Common Mistakes / Traps

- LC 287 (Find Duplicate): Treat array as linked list (value = next index) — same Floyd's trick!
- Don't confuse: detect cycle (LC 141) vs find start (LC 142) — they're different!
- Phase 2 ONLY runs if Phase 1 finds a meeting point — don't mix phases
- The math proof uses ' $F = nC - K$ ' — be ready to explain this to Google interviewers

■ CP / Contest Notes

- In functional graphs (each node has exactly one outgoing edge), Floyd's finds the cycle
- LC 287: 'Find duplicate in array [1..N]' is Floyd's in disguise — classic contest trick
- 30-sec ID: 'No extra space + detect cycle' = Floyd's immediately

■ Similar Problems

#	Problem	Difficulty	Pattern
1	141. Linked List Cycle	Easy	Fast & Slow
2	142. Linked List Cycle II	Medium	Floyd's Algorithm
3	287. Find the Duplicate Number	Medium	Floyd's on Array
4	457. Circular Array Loop	Medium	Fast & Slow

Pattern 06: Merge Technique — Sort & Merge Lists

Metric	Rating
FAANG Frequency	■■■■■ Very High
LeetCode Importance	■■■■■ Essential
Codeforces Usefulness	■■■■ High
Companies	Amazon, Google, Meta, Microsoft
Problems	LC 21, 23, 148, 143

■ Intuition

Merging two sorted lists is the same as the merge step in merge sort. You compare the heads of both lists, pick the smaller one, advance that list. Use a dummy node to avoid head-change problems. For sorting a linked list (LC 148), use merge sort: find middle → sort each half → merge.

■ C++ Code — Sort Linked List (LC 148)

```

// Full Merge Sort on Linked List – O(N log N) time, O(log N) space (recursion)
ListNode* sortList(ListNode* head) {
    if (!head || !head->next) return head;
    // Step 1: Find middle (split into two halves)
    ListNode* mid = getMiddle(head);
    ListNode* right = mid->next;
    mid->next = nullptr; // disconnect left and right halves
    // Step 2: Recursively sort both halves
    ListNode* left = sortList(head);
    right = sortList(right);
    // Step 3: Merge the two sorted halves
    return merge(left, right);
}

ListNode* getMiddle(ListNode* head) {
    ListNode* slow = head;
    ListNode* fast = head->next; // fast starts 1 ahead for left-biased mid
    while (fast && fast->next) {
        slow = slow->next;
        fast = fast->next->next;
    }
    return slow;
}

ListNode* merge(ListNode* l1, ListNode* l2) {
    ListNode dummy(0);
    ListNode* tail = &dummy;
    while (l1 && l2) {
        if (l1->val <= l2->val) { tail->next = l1; l1 = l1->next; }
        else { tail->next = l2; l2 = l2->next; }
        tail = tail->next;
    }
    tail->next = l1 ? l1 : l2;
    return dummy.next;
}

// Time: O(N log N) Space: O(log N) stack

```

■ C++ Code — Merge K Sorted Lists (LC 23)

```
// Use priority queue (min-heap) for efficient K-way merge
ListNode* mergeKLists(vector<ListNode*>& lists) {
    // Min-heap: (value, node pointer)
    auto cmp = [](ListNode* a, ListNode* b){ return a->val > b->val; };
    priority_queue<ListNode*, vector<ListNode*>, decltype(cmp)> pq(cmp);

    for (auto head : lists)
        if (head) pq.push(head);

    ListNode dummy(0);
    ListNode* tail = &dummy;

    while (!pq.empty()) {
        ListNode* node = pq.top(); pq.pop();
        tail->next = node;
        tail = tail->next;
        if (node->next) pq.push(node->next);
    }

    return dummy.next;
}

// Time: O(N log K) Space: O(K) where K = number of lists
```

■ Common Mistakes / Traps

- Sort LL: Don't convert to array, sort, rebuild — that's $O(N)$ extra space, interviewer won't like it
- getMiddle: init fast = head->next (NOT head) for left-biased split — prevents infinite recursion
- Merge K lists brute force (merge two at a time serially) is $O(NK)$ — very slow, avoid
- Always disconnect the two halves (mid->next = nullptr) before recursing

■ Interview Tips

- LC 148 is a classic 'show me you know merge sort' question at Google/Meta
- Explain: 'I'll use top-down merge sort — find mid, split, recurse, merge'
- Follow-up: 'Do it iteratively (bottom-up merge sort)?' → Merge in widths 1, 2, 4, 8...

■ Similar Problems

#	Problem	Difficulty	Pattern
1	21. Merge Two Sorted Lists	Easy	Merge + Dummy Node
2	23. Merge K Sorted Lists	Hard	Merge + Priority Queue
3	148. Sort List	Medium	Merge Sort on LL
4	88. Merge Sorted Array	Easy	Merge Technique (Array)

Pattern 07: K-Group Reversal

Metric	Rating
FAANG Frequency	■■■■■ High
LeetCode Importance	■■■■■■ Hard Must-Know
Codeforces Usefulness	■■■ Moderate

Companies	Google, Meta, Amazon — commonly asked as Hard problem
Problems	LC 25, 24, 92

■ Intuition

You need to reverse every K nodes as a group. Key insight: check if K nodes exist before reversing (otherwise leave as-is). For each group: 1) find the Kth node, 2) disconnect the group, 3) reverse it, 4) reconnect. A dummy node anchors the connection between groups.

■ Dry Run — K=3 Reversal

```
List: 1 -> 2 -> 3 -> 4 -> 5 -> NULL, k=3

dummy -> 1 -> 2 -> 3 -> 4 -> 5 -> NULL
^groupStart

Group 1: [1,2,3] – 3 nodes exist ✓
Find kth node from groupStart = node 3
Save: nextGroup = node 4
Disconnect: node3->next = NULL
Reverse [1->2->3->NULL] => [3->2->1->NULL]
Connect: dummy->next = 3 (new head of group)
node1->next = nextGroup = 4
Now: dummy->3->2->1->4->5->NULL
groupStart moves to node 4

Group 2: [4,5] – only 2 nodes, k=3 not reached → leave as-is

Final: 3 -> 2 -> 1 -> 4 -> 5 -> NULL ✓
```

■ C++ Code — Reverse Nodes in K-Group (LC 25)

```

ListNode* reverseKGroup(ListNode* head, int k) {
    ListNode dummy(0, head);
    ListNode* groupPrev = &dummy;
    while (true) {
        // Check if k nodes exist from current position
        ListNode* kth = getKth(groupPrev, k);
        if (!kth) break; // fewer than k nodes remain
        ListNode* groupNext = kth->next;
        ListNode* curr = groupPrev->next;
        // Reverse the k-node group
        ListNode* prev = groupNext; // tail of next group
        while (curr != groupNext) {
            ListNode* nxt = curr->next;
            curr->next = prev;
            prev = curr;
            curr = nxt;
        }
        // Reconnect: groupPrev -> reversed group head
        ListNode* tmp = groupPrev->next; // this is old head = new tail
        groupPrev->next = kth;           // kth is new head
        groupPrev = tmp;                 // advance groupPrev
    }
    return dummy.next;
}

ListNode* getKth(ListNode* curr, int k) {
    while (curr && k > 0) { curr = curr->next; k--; }
    return curr;
}

// Time: O(N) Space: O(1)

```

■ Common Mistakes / Traps

- Don't forget to check K nodes exist before reversing — the remainder stays as-is
- The old group head becomes the new group tail — don't lose this pointer
- groupPrev must advance to the old head (new tail) for next iteration
- LC 24 (Swap Pairs) is K=2 case of this — practice this simpler version first

■ Interview Tips

- This is a Hard problem — if you get it right in interview, you likely get the offer
- Break the problem: 'First, let me solve reverse of one group, then repeat'
- Interviewer follow-up: 'What if you reverse only even-indexed groups?' → LC 2074

Pattern 08: Linked List + Stack

Metric	Rating
FAANG Frequency	■■■ Moderate
LeetCode Importance	■■■■ Important
Codeforces Usefulness	■■■ Moderate
Companies	Amazon, Microsoft, Adobe
Problems	LC 234, 445, 1019, 2130

■ Intuition

Stack gives you LIFO (Last In First Out) — i.e., reverse order. When you push all linked list nodes onto a stack and pop them, you're traversing the list in reverse. Use this when you need to process from tail to head, compare forward vs backward, or handle parentheses-like nesting.

■ C++ Code — Add Two Numbers II (LC 445)

```
// Numbers stored most-significant digit first
// Use stacks to process from least significant digit
ListNode* addTwoNumbers(ListNode* l1, ListNode* l2) {
    stack<int> s1, s2;
    while (l1) { s1.push(l1->val); l1 = l1->next; }
    while (l2) { s2.push(l2->val); l2 = l2->next; }
    int carry = 0;
    ListNode* head = nullptr;
    while (!s1.empty() || !s2.empty() || carry) {
        int sum = carry;
        if (!s1.empty()) { sum += s1.top(); s1.pop(); }
        if (!s2.empty()) { sum += s2.top(); s2.pop(); }
        carry = sum / 10;
        ListNode* node = new ListNode(sum % 10);
        node->next = head;    // prepend (build in reverse)
        head = node;
    }
    return head;
}
// Time: O(N+M) Space: O(N+M)
```

■ C++ Code — Next Greater Node (LC 1019)


```

vector<int> nextLargerNodes(ListNode* head) {
    vector<int> vals;
    while (head) { vals.push_back(head->val); head = head->next; }
    int n = vals.size();
    vector<int> result(n, 0);
    stack<int> mono; // monotonic stack (decreasing), stores indices
    for (int i = 0; i < n; i++) {
        while (!mono.empty() && vals[mono.top()] < vals[i]) {
            result[mono.top()] = vals[i];
            mono.pop();
        }
        mono.push(i);
    }
    return result;
}
// Time: O(N) Space: O(N)

```

■ Common Mistakes / Traps

- LC 234 Palindrome: Stack approach uses $O(N)$ space — interviewer will push for $O(1)$ (reverse half)
- Stack = extra space, so use only when $O(1)$ is NOT required
- Monotonic stack on LL: convert to array first for indexed access

■ Interview Tips

- If asked to compare LL in reverse without reversing: push to stack, then compare
- Next Greater Element on LL = classic monotonic stack — know this pattern well

Pattern 09: Recursion on Linked List

Metric	Rating
FAANG Frequency	■■■■■ High
LeetCode Importance	■■■■■ Important
Codeforces Usefulness	■■■ Moderate
Companies	Google, Meta — tests deep understanding of recursion
Problems	LC 206, 25, 234, 24, 2487

■ Intuition

Recursion on LL = 'trust the function to solve the smaller problem, just fix the current node.' The call stack inherently reverses traversal order. Key mental model: define clearly what your function returns, then use that return value to fix `curr->next`.

■ Recursive Thinking Framework

Recursive LL Template

- **Base case:** Empty node or single node — return it directly
- **Recursive call:** Solve for `head->next` (trust it's correct)
- **Fix current:** Use the return value to connect current node
- **Return:** Return what the new head should be

■ C++ Code — Remove Nodes Greater Than Right (LC 2487)

```
// Remove all nodes where node.val < any node to its right
// Recursion naturally processes right-to-left
ListNode* removeNodes(ListNode* head) {
    if (!head) return nullptr;
    // Recursively fix the rest
    head->next = removeNodes(head->next);
    // If next node is greater, current node should be removed
    if (head->next && head->val < head->next->val)
        return head->next; // skip current
    return head;
}
// Time: O(N) Space: O(N) stack
// Elegance: recursive reversal makes right-to-left processing trivial
```

■ C++ Code — Recursive Palindrome Check (LC 234)

```
// Global pointer for left side comparison
ListNode* left;

bool check(ListNode* right) {
    if (!right) return true;
    if (!check(right->next)) return false;
    // Now we're at the rightmost node, comparing with left
    bool ok = (left->val == right->val);
    left = left->next; // advance left
    return ok;
}

bool isPalindrome(ListNode* head) {
    left = head;
    return check(head);
}

// Time: O(N) Space: O(N) — Note: O(1) space solution uses reverse
```

■ Common Mistakes / Traps

- Space is $O(N)$ due to call stack — always mention this vs iterative $O(1)$ space
- Recursion depth = list length — for $N=10^5$, may cause stack overflow in interviews!
- For production code: use iterative approach. For interviews: recursive shows understanding.
- Be clear: 'recursive approach uses $O(N)$ implicit stack space'

■ Interview Tips

- Recursive reverse is beautiful code but $O(N)$ space — know BOTH iterative and recursive
- LC 2487 is a great example where recursion is more elegant than iteration
- Google loves asking 'can you do it recursively and iteratively both?'

Pattern 10: In-Place Manipulation — Reorder & Partition

Metric	Rating
FAANG Frequency	■■■■■ Very High
LeetCode Importance	■■■■■ Essential Combo Pattern
Codeforces Usefulness	■■■■ High
Companies	Meta, Amazon, Google — tests combination of patterns
Problems	LC 143, 86, 328, 234, 61

■ Intuition

Reorder List (LC 143) is the ultimate combo problem: **find middle** (P01) + **reverse second half** (P02) + **merge two lists** (P03/P06). Partition List (LC 86) uses two pointers/dummy nodes to split into two halves then reconnect.

■ Dry Run — Reorder List (LC 143)

```
Input: 1 -> 2 -> 3 -> 4 -> 5 -> NULL
Output: 1 -> 5 -> 2 -> 4 -> 3 -> NULL

Step 1: Find middle using slow/fast
slow=3 (middle node)

Step 2: Split: left = 1->2->3, right = 4->5
(set mid->next = NULL)

Step 3: Reverse right half
right: 4->5->NULL => 5->4->NULL

Step 4: Merge left and right alternately
l=1, r=5: 1->5, advance l to 2, r to 4
l=2, r=4: 1->5->2->4, advance l to 3, r=NULL
l=3, r=NULL: 1->5->2->4->3

Result: 1->5->2->4->3->NULL ✓
```

■ C++ Code — Reorder List (LC 143)

```

void reorderList(ListNode* head) {
    if (!head || !head->next) return;
    // Step 1: Find middle
    ListNode* slow = head, *fast = head;
    while (fast->next && fast->next->next) {
        slow = slow->next;
        fast = fast->next->next;
    }
    // Step 2: Reverse second half
    ListNode* second = slow->next;
    slow->next = nullptr; // split the list
    ListNode* prev = nullptr;
    while (second) {
        ListNode* nxt = second->next;
        second->next = prev;
        prev = second;
        second = nxt;
    }
    second = prev; // head of reversed second half
    // Step 3: Merge two halves alternately
    ListNode* first = head;
    while (second) {
        ListNode* tmp1 = first->next;
        ListNode* tmp2 = second->next;
        first->next = second;
        second->next = tmp1;
        first = tmp1;
        second = tmp2;
    }
}
// Time: O(N) Space: O(1)

```

■ C++ Code — Partition List (LC 86)

```
// All nodes < x come before nodes >= x
ListNode* partition(ListNode* head, int x) {
    ListNode lessHead(0), greaterHead(0);
    ListNode* less = &lessHead;
    ListNode* greater = &greaterHead;
    while (head) {
        if (head->val < x) {
            less->next = head;
            less = less->next;
        } else {
            greater->next = head;
            greater = greater->next;
        }
        head = head->next;
    }
    greater->next = nullptr;    // important! prevent cycle
    less->next = greaterHead.next;
    return lessHead.next;
}
// Time: O(N) Space: O(1)
```

■ Common Mistakes / Traps

- LC 143: When N is odd, the middle node goes to the LEFT half — slow stops at true middle
- LC 86: MUST set greater->next = nullptr at the end — forgetting this creates a cycle!
- Merge step in LC 143: second list may be shorter — stop when second is NULL
- Always disconnect the two halves (slow->next = nullptr) before reversing

■ Interview Tips

- LC 143 tests 3 patterns in one — this is why Meta/Amazon love it
- Explain your approach: 'I'll find the middle, reverse the second half, then merge'
- Rejected if: you use O(N) extra space (vector/array). Must be O(1).
- LC 328 (Odd Even LL): same partition idea but by position, not value

■ Similar Problems

#	Problem	Difficulty	Pattern
1	143. Reorder List	Medium	Fast&Slow; + Reverse + Merge
2	86. Partition List	Medium	Two Dummy Nodes
3	328. Odd Even Linked List	Medium	In-Place Partition
4	234. Palindrome Linked List	Easy	Fast&Slow; + Reverse

5	148. Sort List	Medium	Merge Sort
---	----------------	--------	------------

■ Learning Roadmap & Revision Guide

■ Week-by-Week Plan for Beginners

Week	Patterns	Key Problems	Goal
Week 1	P01 Fast&Slow, P03 Dummy Node	LC 141, 876, 21, 203	Pointer mechanics
Week 2	P02 Reverse LL, P04 Two Ptr	LC 206, 92, 19, 61	Pointer rewiring
Week 3	P05 Floyd, P06 Merge	LC 142, 148, 23	Math + Sorting
Week 4	P07 K-Group, P10 In-Place	LC 25, 143, 86	Combo patterns
Week 5	P08 Stack, P09 Recursion	LC 445, 234, 2487	Advanced tools
Week 6	All patterns	Leetcode LL Tag	Speed + accuracy

■ Must-Solve LeetCode Problems (Ordered by Importance)

#	Problem	Difficulty	Pattern
1	206. Reverse Linked List	Easy	P02 — Foundation
2	141. Linked List Cycle	Easy	P01/P05
3	21. Merge Two Sorted Lists	Easy	P03/P06
4	876. Middle of Linked List	Easy	P01
5	19. Remove Nth from End	Medium	P04
6	142. Linked List Cycle II	Medium	P05 Floyd's
7	92. Reverse Linked List II	Medium	P02
8	143. Reorder List	Medium	P01+P02+P06
9	148. Sort List	Medium	P06 Merge Sort
10	86. Partition List	Medium	P10
11	82. Remove Duplicates II	Medium	P03 Dummy
12	328. Odd Even Linked List	Medium	P10
13	23. Merge K Sorted Lists	Hard	P06 + Heap
14	25. Reverse Nodes in K-Group	Hard	P07
15	287. Find Duplicate Number	Medium	P05 Floyd on Array

■ Pattern Dependency Map

Learn in This Order for Maximum Efficiency

- P01 Fast&Slow; → needed for P05 Floyd, P10 Reorder, P06 Sort
- P02 Reverse → needed for P07 K-Group, P10 Reorder, P09 Recursion
- P03 Dummy Node → used inside P04, P06, P07, P10 always
- P05 Floyd → extends P01 (same pointers, different math)
- P10 In-Place → combines P01 + P02 + P06 (the final boss combo)
- P07 K-Group → extends P02 (reverse applied repeatedly with bookkeeping)

■ Universal Problem-Solving Framework

The 7-Step Interview/Contest Thinking Process

- **Step 1 — Read keywords:** cycle/loop → Floyd, middle → fast&slow;, reverse → P02, Kth-end → two-ptr
- **Step 2 — Ask constraints:** O(1) space? → no stack/array. Single pass? → two pointers with gap.
- **Step 3 — Draw the list:** 1→2→3→NULL. Physically draw pointer movements on paper.
- **Step 4 — Brute first:** Even if inefficient, state it. Shows you understand the problem.
- **Step 5 — Optimize:** Identify the bottleneck (extra space? two passes?) and eliminate.
- **Step 6 — Edge cases ALWAYS:** empty list, single node, two nodes, all same values, cycle.
- **Step 7 — Communicate:** 'I see this uses fast-slow because...' shows pattern recognition.

■ FAANG vs OA vs Codeforces Priority

Context	Most Important Patterns	Least Important
FAANG Interview	P02 Reverse, P05 Floyd, P10 Combo, P07 K-Group	P08 Stack (basic)
OA / Online Assess.	P01 Fast&Slow;, P02 Reverse, P04 Two-Ptr, P06 Merge	P09 Recursion
Codeforces Div2	P05 Floyd (functional graph), P06 Merge Sort	P03 Dummy (too easy)

The goal is NOT to memorize solutions. It is to recognize patterns instantly and adapt them.

When you see a new problem, ask: What does this remind me of? Which pattern fits?